SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.* (BL2, BL3)

Activity Tool : Case Study (Medium)
Assessment Tool Weightage: 30%

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	**5**
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	**5**
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	**5**
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	**5**
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	**5**
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	**5**

7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	**5**
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	**5**
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	**5**
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	**5**

Code of Conduct:

I P.Harsha vardhan (192525464) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. *I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning

Solution Design	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Question 1

Online shopping platform - Normalize Order to 3NF

A:

Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

Step 1 - Functional Dependencies Identified

- OrderID -> CustID
- CustID -> CustName
- ProdID -> ProdName, Price
- (OrderID, ProdID) -> Qty

Candidate Key

Candidate key = (OrderID, ProdID), since one order can contain many products and Qty is specific to an order-product combination.

Anomalies in the

- Insertion anomaly: a new product cannot be added to the catalog until it is part of an order.
- Update anomaly: if a customer's name changes, every order row for that customer must be updated.
- Deletion anomaly: deleting the only order of a customer erases that customer's details entirely.

Step 2 - Normalization

1NF

All attributes already hold atomic (single) values, so the relation is in 1NF.

2NF

The key is composite (OrderID, ProdID). CustID and CustName depend only on OrderID (part of the key), and ProdName/Price depend only on ProdID (part of the key) - these are partial dependencies, which 2NF disallows.

Remove the partial dependencies into their own relations.

3NF

After the 2NF split, Customer(CustID -> CustName) and Product(ProdID -> ProdName, Price) each have a single-attribute key with no transitive dependency left, so the design is already in 3NF.

Final Normalized Schema

Customer(CustID, CustName)
Product(ProdID, ProdName, Price)
Orders(OrderID, CustID) -- FK CustID -> Customer
OrderDetails(OrderID, ProdID, Qty) -- FK OrderID -> Orders, FK ProdID -> Product

SQL Output (executed result)

```
mysql> USE q1_orders;

+--------+------------+
| CustID | CustName   |
+--------+------------+
|      1 | Arun Kumar |
|      2 | Divya S    |
+--------+------------+
+--------+----------------+--------+
| ProdID | ProdName       | Price  |
+--------+----------------+--------+
|    101 | USB Cable      | 150.00 |
|    102 | Wireless Mouse | 699.00 |
+--------+----------------+--------+
+---------+--------+
| OrderID | CustID |
+---------+--------+
|    5001 |      1 |
|    5002 |      2 |
+---------+--------+
+---------+--------+-----+
| OrderID | ProdID | Qty |
+---------+--------+-----+
|    5001 |    101 |   2 |
|    5001 |    102 |   1 |
|    5002 |    101 |   1 |
+---------+--------+-----+
```

Question 2

Hospital system - Anomalies and normalize Patient to BCNF

ANS:

Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

Step 1 - Functional Dependencies Identified

- PID -> PName, DocID, WardNo
- DocID -> DocName, Specialization
- WardNo -> WardName

Candidate Key

Candidate key = PID.

Anomalies in the Given Relation

- Update anomaly: changing a doctor's specialization requires updating every patient row treated by that doctor.
- Insertion anomaly: a newly joined doctor cannot be recorded until assigned at least one patient.
- Deletion anomaly: discharging the last patient in a ward deletes the ward's name from the database.

Step 2 - Normalization

1NF -> 2NF -> 3NF

All attributes are atomic and depend fully on the single-attribute key PID, so the relation already satisfies 1NF, 2NF and 3NF.

BCNF check

BCNF requires every determinant to be a candidate key.

DocID -> DocName, Specialization: DocID is a determinant but not a candidate key of Patient - violates BCNF.

WardNo -> WardName: WardNo is a determinant but not a candidate key of Patient - violates BCNF.

Decompose to remove both violations.

Final Normalized Schema

Doctor(DocID, DocName, Specialization)
Ward(WardNo, WardName)
Patient(PID, PName, DocID, WardNo) -- FK DocID -> Doctor, FK WardNo -> Ward

SQL Output (executed result)

```
mysql> USE q2_hospital;

+-------+-----------+----------------+
| DocID | DocName   | Specialization |
+-------+-----------+----------------+
|   201 | Dr. Meera | Cardiology     |
|   202 | Dr. Ravi  | Orthopedics    |
+-------+-----------+----------------+
+--------+--------------+
| WardNo | WardName     |
+--------+--------------+
|      1 | General Ward |
|      2 | ICU          |
+--------+--------------+
+-----+----------+-------+--------+
| PID | PName    | DocID | WardNo |
+-----+----------+-------+--------+
|   1 | Suresh K |   201 |      1 |
|   2 | Priya R  |   201 |      2 |
|   3 | Kavya M  |   202 |      1 |
+-----+----------+-------+--------+
```

Question 3

University enrollment - Analyze FDs and apply 2NF

ANS:

Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade)

Step 1 - Functional Dependencies Identified

- SID -> SName
- CourseID -> CourseName, Faculty
- (SID, CourseID) -> Grade

Candidate Key

Candidate key = (SID, CourseID), since a student enrolls in many courses and a course has many students; Grade depends on the combination.

Anomalies in the Given Relation

- Redundancy: a student's name is repeated once per course they enroll in.
- Update anomaly: correcting a course's faculty name means updating it in every enrolled student's row.
- Deletion anomaly: removing the last enrollment of a course erases that course's name and faculty entirely.

Step 2 - Normalization

1NF

All attribute values are atomic, so the relation is in 1NF.

2NF

SName depends only on SID (part of the key) and CourseName/Faculty depend only on CourseID (part of the key) - both are partial dependencies on the composite key.

Split the partially dependent attributes into their own relations, leaving only the fully key-dependent attribute (Grade) in the linking relation.

Final Normalized Schema

Student(SID, SName)
Course(CourseID, CourseName, Faculty)
Enrollment(SID, CourseID, Grade) -- FK SID -> Student, FK CourseID -> Course

SQL Output (executed result)

```
mysql> USE q3_university;

+-----+----------+
| SID | SName    |
+-----+----------+
|   1 | Naveen T |
|   2 | Sneha P  |
+-----+----------+
+----------+------------+-------------+
| CourseID | CourseName | Faculty     |
+----------+------------+-------------+
|      301 | DBMS       | Dr. Anand   |
|      302 | OS         | Dr. Lakshmi |
+----------+------------+-------------+
+-----+----------+-------+
| SID | CourseID | Grade |
+-----+----------+-------+
|   1 |      301 | A     |
|   1 |      302 | B+    |
|   2 |      301 | A-    |
+-----+----------+-------+
```

Question 4

Airline booking - Normalize Booking to BCNF

ANS:

Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

Step 1 - Functional Dependencies Identified

- FlightNo -> DeptCity, ArrCity
- PassID -> PassName
- (FlightNo, Date, PassID) -> Seat

Candidate Key

Candidate key = (FlightNo, Date, PassID), since the same flight number flies on many dates and carries many passengers, each assigned one seat per specific flight instance.

Anomalies in the Given Relation

- Insertion anomaly: a new flight route cannot be stored until at least one passenger books it.
- Update anomaly: changing a flight's arrival city means updating it on every booking row for that flight.
- Deletion anomaly: cancelling the only booking on a date removes the flight's route information.

Step 2 - Normalization

1NF - 3NF

All attributes are atomic; the relation reaches 3NF but still has non-key determinants.

BCNF check

FlightNo -> DeptCity, ArrCity: FlightNo alone is not a candidate key (the key needs Date and PassID too) - violates BCNF.

PassID -> PassName: PassID alone is not a candidate key - violates BCNF.

Decompose to isolate both violating dependencies.

Final Normalized Schema

Flight(FlightNo, DeptCity, ArrCity)
Passenger(PassID, PassName)
Booking(FlightNo, Date, PassID, Seat) -- FK FlightNo -> Flight, FK PassID -> Passenger

SQL Output (executed result)

```
mysql> USE q4_airline;

+----------+----------+---------+
| FlightNo | DeptCity | ArrCity |
+----------+----------+---------+
| AI501    | Chennai  | Delhi   |
| AI502    | Delhi    | Mumbai  |
+----------+----------+---------+
+--------+-----------+
| PassID | PassName  |
+--------+-----------+
|      1 | Vignesh S |
|      2 | Anitha R  |
+--------+-----------+
+----------+------------+--------+------+
| FlightNo | BookDate   | PassID | Seat |
+----------+------------+--------+------+
| AI501    | 2026-08-10 |      1 | 12A  |
| AI501    | 2026-08-10 |      2 | 12B  |
+----------+------------+--------+------+
```

Question 5

Multinational HR system - Identify FDs/transitive dependencies, normalize to 3NF

ANS:

Employee(EmpID, EmpName, DeptID, DeptName, DeptLocation, Country, ManagerID, Salary)

Step 1 - Functional Dependencies Identified

- EmpID -> EmpName, DeptID, ManagerID, Salary
- DeptID -> DeptName, DeptLocation, Country
- Transitive dependency: EmpID -> DeptID -> DeptName, DeptLocation, Country

Candidate Key

Candidate key = EmpID.

Anomalies in the Given Relation

- Redundancy: department location and country repeat on every employee row in that department.
- Update anomaly: if a department relocates, every employee row for that department must be updated.
- Insertion anomaly: a newly opened department cannot be recorded until an employee is hired into it.

Step 2 - Normalization

1NF - 2NF

All attributes are atomic and the key is a single attribute (EmpID), so partial dependency cannot occur - the relation is in 2NF.

3NF

DeptName, DeptLocation and Country depend on DeptID, which itself depends on EmpID - a transitive dependency, which 3NF disallows.

Remove the transitively dependent attributes into a separate Department relation.

Final Normalized Schema

Department(DeptID, DeptName, DeptLocation, Country)
Employee(EmpID, EmpName, DeptID, ManagerID, Salary) -- FK DeptID -> Department, FK ManagerID -> Employee (self-reference)

SQL Output (executed result)

```
mysql> USE q5_hr;

+--------+-------------+--------------+-----------+
| DeptID | DeptName    | DeptLocation | Country   |
+--------+-------------+--------------+-----------+
|     10 | Engineering | Bengaluru    | India     |
|     20 | Sales       | Singapore    | Singapore |
+--------+-------------+--------------+-----------+
+-------+-----------+--------+-----------+-----------+
| EmpID | EmpName   | DeptID | ManagerID | Salary    |
+-------+-----------+--------+-----------+-----------+
|     1 | Karthik V |     10 |      NULL | 120000.00 |
|     2 | Fatima N  |     10 |         1 |  85000.00 |
|     3 | John Lee  |     20 |      NULL |  95000.00 |
+-------+-----------+--------+-----------+-----------+
```

Question 6

Library system - Identify MVDs and decompose to 4NF

ANS:

Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

Step 1 - Functional Dependencies Identified

- MemberID -> MemberName
- BookID -> Title
- Multi-valued dependency: BookID ->> Author (a book can have several authors, independent of genre)
- Multi-valued dependency: BookID ->> Genre (a book can belong to several genres, independent of author)

Candidate Key

The relation is already in BCNF with respect to functional dependencies (every determinant above is a candidate key fragment), but it still suffers redundancy because of the two independent multi-valued dependencies on BookID.

Anomalies in the Given Relation

- Redundancy: a book with 2 authors and 2 genres needs 4 rows to represent every author-genre combination, even though authors and genres are unrelated to each other.
- Insertion anomaly: adding one more genre for a book forces duplicating a row for every existing author.
- Deletion anomaly: removing one author's row can accidentally remove genre information if not handled carefully.

Step 2 - Normalization

Identify the MVDs

BookID ->> Author | Genre : for a fixed BookID, the set of Authors is independent of the set of Genres, which is the defining condition of a nontrivial multi-valued dependency.

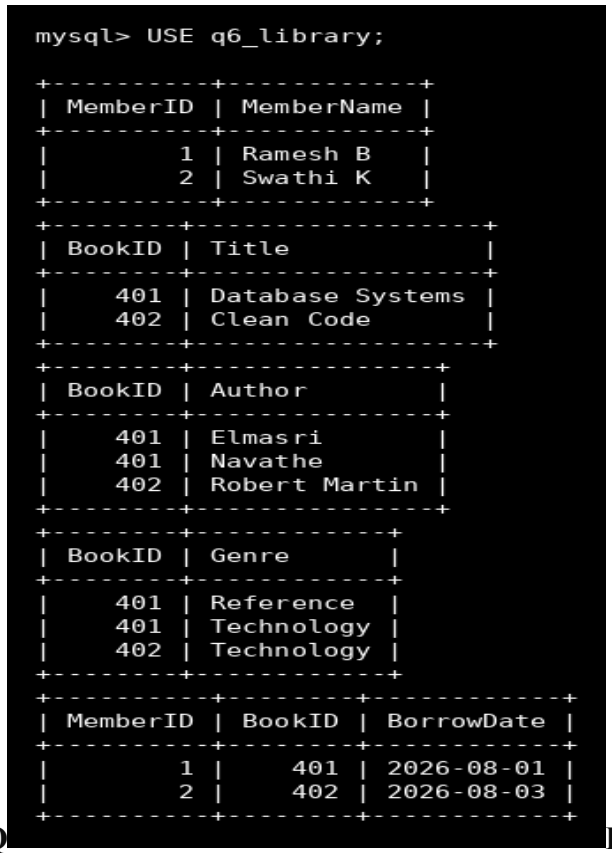
4NF decomposition

Split the relation so each independent multi-valued fact gets its own relation, eliminating the need to store cross-product combinations.

Final Normalized Schema

Member(MemberID, MemberName)
Book(BookID, Title)
BookAuthor(BookID, Author)
BookGenre(BookID, Genre)
Borrow(MemberID, BookID, BorrowDate) -- FK MemberID -> Member, FK BookID -> Book

SQL Output (executed result)



```
mysql> USE q6_library;

+----------+------------+
| MemberID | MemberName |
+----------+------------+
|        1 | Ramesh B   |
|        2 | Swathi K   |
+----------+------------+
+--------+------------------+
| BookID | Title            |
+--------+------------------+
|    401 | Database Systems |
|    402 | Clean Code       |
+--------+------------------+
+--------+----------------+
| BookID | Author         |
+--------+----------------+
|    401 | Elmasri        |
|    401 | Navathe        |
|    402 | Robert Martin  |
+--------+----------------+
+--------+------------+
| BookID | Genre      |
+--------+------------+
|    401 | Reference  |
|    401 | Technology |
|    402 | Technology |
+--------+------------+
+----------+--------+------------+
| MemberID | BookID | BorrowDate |
+----------+--------+------------+
|        1 |    401 | 2026-08-01 |
|        2 |    402 | 2026-08-03 |
+----------+--------+------------+
```

Question 7

Telecom billing - Identify candidate keys, primary key, and FDs before normalization

ANS:

Bill(BillNo, CustID, CustName, PhoneNo, PlanID, PlanName, PlanCost, CallDate, Duration, Amount)

Step 1 - Functional Dependencies Identified

- BillNo -> CustID, PlanID, CallDate, Duration, Amount
- CustID -> CustName, PhoneNo
- PhoneNo -> CustID (phone number is also unique per customer)
- PlanID -> PlanName, PlanCost

Candidate Key

Candidate keys: {BillNo} and {PhoneNo, CallDate} (a phone number can only generate one bill per call date in this design). Primary key chosen: BillNo, since it is the simplest single-attribute key that determines every other attribute. PhoneNo is therefore a candidate/alternate key via CustID.

Anomalies in the Given Relation

- Redundancy: customer name and phone repeat on every bill row for that customer.
- Redundancy: plan name and cost repeat on every bill row that uses that plan.

Step 2 - Normalization

Normalize to 3NF (for completeness)

CustID -> CustName, PhoneNo is a transitive dependency through BillNo, and PlanID -> PlanName, PlanCost is transitive as well.

Remove both transitively dependent groups into their own relations.

Final Normalized Schema

Customer(CustID, CustName, PhoneNo)
Plan(PlanID, PlanName, PlanCost)
Bill(BillNo, CustID, PlanID, CallDate, Duration, Amount) -- FK CustID -> Customer, FK PlanID -> Plan

SQL Output (executed result)

```
mysql> USE q7_telecom;

+--------+----------+------------+
| CustID | CustName | PhoneNo    |
+--------+----------+------------+
|      1 | Harish P | 9876500001 |
|      2 | Deepa V  | 9876500002 |
+--------+----------+------------+
+--------+-----------------+----------+
| PlanID | PlanName        | PlanCost |
+--------+-----------------+----------+
|      1 | Prepaid Basic   |   199.00 |
|      2 | Postpaid Family |   599.00 |
+--------+-----------------+----------+
+--------+--------+--------+------------+----------+--------+
| BillNo | CustID | PlanID | CallDate   | Duration | Amount |
+--------+--------+--------+------------+----------+--------+
|   9001 |      1 |      1 | 2026-08-01 |       12 |  15.00 |
|   9002 |      2 |      2 | 2026-08-02 |       45 |  50.00 |
+--------+--------+--------+------------+----------+--------+
```

Question 8

Retail inventory - Apply complete normalization to Product

ANS:

Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

Step 1 - Functional Dependencies Identified

- PID -> PName, SupplierID, Category, Price, Warehouse
- SupplierID -> SupplierName

Candidate Key

Candidate key = PID.

Anomalies in the Given Relation

- Redundancy: the supplier's name is repeated on every product row supplied by that supplier.
- Update anomaly: a supplier's name change must be updated across every one of its products.
- Insertion anomaly: a new supplier cannot be added to the database until it supplies at least one product.

Step 2 - Normalization

1NF - 2NF

The key is a single attribute (PID), so no partial dependency can exist - the relation is already in 2NF.

3NF / BCNF

SupplierID -> SupplierName is a transitive dependency (PID -> SupplierID -> SupplierName), which violates 3NF.

Remove it into its own Supplier relation.

Final Normalized Schema

Supplier(SupplierID, SupplierName)
Product(PID, PName, SupplierID, Category, Price, Warehouse) -- FK SupplierID -> Supplier

Justification:

After decomposition every determinant (PID in Product, SupplierID in Supplier) is a candidate key, so the design satisfies both 3NF and BCNF - this is the complete normalization.

SQL Output (executed result)

```
mysql> USE q8_retail;

+------------+--------------------+
| SupplierID | SupplierName       |
+------------+--------------------+
|          1 | Sri Lakshmi Traders |
|          2 | Global Parts Co    |
+------------+--------------------+
+-----+------------------+------------+-------------+--------+---------------+
| PID | PName            | SupplierID | Category    | Price  | Warehouse     |
+-----+------------------+------------+-------------+--------+---------------+
| 501 | LED Bulb 9W      |          1 | Electricals | 120.00 | WH-Chennai    |
| 502 | Ball Bearing 6mm |          2 | Hardware    |  45.00 | WH-Coimbatore |
+-----+------------------+------------+-------------+--------+---------------+
```

Question 9

School ERP - Propose a fully normalized design up to BCNF with justification

ANS:

SchoolERP(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, Subject, Marks, ParentName, ParentContact)

Step 1 - Functional Dependencies Identified

- StudentID -> StudentName, ClassID, ParentName, ParentContact
- ClassID -> ClassName
- (ClassID, Subject) -> TeacherID
- TeacherID -> TeacherName
- (StudentID, Subject) -> Marks

Candidate Key

No single attribute or the original combination determines every column at once - the schema as given is a poorly-designed, over-wide relation combining three separate real-world entities (student, class/subject/teacher assignment, and marks).

Anomalies in the Given Relation

- Redundancy: class name repeats for every student in that class, and teacher name repeats for every student they teach.
- Update anomaly: reassigning a teacher to a class-subject requires updating it on every affected student's row.
- Insertion anomaly: a new class-subject-teacher assignment cannot be recorded until a student is enrolled and scored.
- Deletion anomaly: removing a student's only mark row can wipe out the class-subject-teacher assignment if it existed nowhere else.

Step 2 - Normalization

Decompose by dependency group

Group 1 (StudentID -> ...): student's own attributes go into Student.

Group 2 (ClassID -> ClassName): class attributes go into Class.

Group 3 (TeacherID -> TeacherName): teacher attributes go into Teacher.

Group 4 ((ClassID, Subject) -> TeacherID): the class-subject-teacher assignment becomes its own relation.

Group 5 ((StudentID, Subject) -> Marks): marks become their own relation.

Final Normalized Schema

Class(ClassID, ClassName)
Teacher(TeacherID, TeacherName)
Student(StudentID, StudentName, ClassID, ParentName, ParentContact) -- FK ClassID -> Class
ClassSubjectTeacher(ClassID, Subject, TeacherID) -- FK ClassID -> Class, FK TeacherID -> Teacher
Marks(StudentID, Subject, Marks) -- FK StudentID -> Student

Justification

Every determinant in the five relations above (StudentID, ClassID, TeacherID, (ClassID,Subject), (StudentID,Subject)) is exactly the primary key of its own relation, with no attribute left partially or transitively dependent - this satisfies BCNF.

SQL Output (executed result)

```
mysql> USE q9_schoolerp;

+---------+-----------+
| ClassID | ClassName |
+---------+-----------+
|       7 | VII-A     |
|       8 | VIII-B    |
+---------+-----------+
+-----------+-------------+
| TeacherID | TeacherName |
+-----------+-------------+
|         1 | Mrs. Geetha |
|         2 | Mr. Arvind  |
+-----------+-------------+
+-----------+-------------+---------+------------+---------------+
| StudentID | StudentName | ClassID | ParentName | ParentContact |
+-----------+-------------+---------+------------+---------------+
|         1 | Yamuna S    |       7 | Selvam R   | 9840011122    |
|         2 | Bala K      |       8 | Kumar M    | 9840033344    |
+-----------+-------------+---------+------------+---------------+
+---------+---------+-----------+
| ClassID | Subject | TeacherID |
+---------+---------+-----------+
|       7 | Maths   |         1 |
|       7 | Science |         2 |
|       8 | Maths   |         2 |
+---------+---------+-----------+
+-----------+---------+-------+
| StudentID | Subject | Marks |
+-----------+---------+-------+
|         1 | Maths   |    88 |
|         1 | Science |    91 |
|         2 | Maths   |    76 |
+-----------+---------+-------+
```

Question 10

Movie streaming platform - Identify join dependency, decompose to 5NF with lossless-join verification

ANS:

Streaming(Movie, Actor, Platform)

Step 1 - Functional Dependencies Identified

- No single functional dependency holds among Movie, Actor, and Platform - the relation is already all-key and in BCNF with respect to FDs.
- Business rule (join dependency): whenever (Movie,Actor) is a valid pairing, (Actor,Platform) is a valid pairing, and (Movie,Platform) is a valid pairing simultaneously, then (Movie,Actor,Platform) must be a valid combination.

Candidate Key

Candidate key = (Movie, Actor, Platform) - the whole relation, since no proper subset determines the rest.

Anomalies in the Given Relation

- Redundancy: storing all three attributes together forces repeating rows for every valid movie-actor-platform triple, even though the constraint is really three independent binary relationships.
- Update anomaly: adding a new platform for an actor who appears in several movies means inserting one new row per movie that actor appears in.

Step 2 - Normalization

Identify the join dependency (JD)

JD = *((Movie,Actor), (Actor,Platform), (Movie,Platform))

This says the relation can be losslessly reconstructed from these three binary projections - a classic candidate for 5NF decomposition (the same pattern as the textbook Supplier-Part-Project example).

Decompose to 5NF

Project the relation onto the three pairwise binary relations named by the JD.

Lossless-join verification

Sample data: MovieActor = {(M1,Nayanthara), (M1,VijaySethupathi)}; ActorPlatform = {(Nayanthara,Netflix), (Nayanthara,PrimeVideo), (VijaySethupathi,Netflix)}; MoviePlatform = {(M1,Netflix), (M1,PrimeVideo)}.

Natural-joining all three back together yields exactly: (M1,Nayanthara,Netflix), (M1,Nayanthara,PrimeVideo), (M1,VijaySethupathi,Netflix).

(M1, VijaySethupathi, PrimeVideo) is correctly NOT produced, because (VijaySethupathi, PrimeVideo) is absent from ActorPlatform - so the 3-way join reproduces only genuine facts with no spurious tuples, confirming the decomposition is lossless and the design satisfies 5NF.

Final Normalized Schema

MovieActor(Movie, Actor)
ActorPlatform(Actor, Platform)
MoviePlatform(Movie, Platform)

SQL Output (executed result)

```
mysql> USE q10_streaming;

+------------------+------------------+
| Movie            | Actor            |
+------------------+------------------+
| Interstellar Two | Nayanthara       |
| Interstellar Two | Vijay Sethupathi |
+------------------+------------------+
+------------------+------------+
| Actor            | Platform   |
+------------------+------------+
| Nayanthara       | Netflix    |
| Nayanthara       | PrimeVideo |
| Vijay Sethupathi | Netflix    |
+------------------+------------+
+------------------+------------+
| Movie            | Platform   |
+------------------+------------+
| Interstellar Two | Netflix    |
| Interstellar Two | PrimeVideo |
+------------------+------------+
+------------------+------------------+------------+
| Movie            | Actor            | Platform   |
+------------------+------------------+------------+
| Interstellar Two | Nayanthara       | Netflix    |
| Interstellar Two | Nayanthara       | PrimeVideo |
| Interstellar Two | Vijay Sethupathi | Netflix    |
+------------------+------------------+------------+
```